ARCHITECTURE & MODEL RESEARCH

AI-Powered Intelligent Query Resolution System

Infosys Springboard Virtual Internship 7.0
Milestone 1 • Proposed System Design

Project focus
A document-grounded assistant that retrieves information from uploaded files and responds with traceable sources.

1. Project Background

The proposed system is a conversational assistant for finding answers inside a collection of user-provided documents. A user uploads files such as PDF, DOCX, TXT, or FAQ CSV documents and then asks a question through a browser interface. The application searches the available content, prepares an answer from the retrieved evidence, and displays the relevant source references.

Potential use cases include student handbooks, internal HR policies, product documentation, and technical guides. The design should make it easier to find information without requiring users to manually scan long documents.

2. Goals and Scope

• Accept common document formats and extract their text for indexing.

• Use Retrieval-Augmented Generation (RAG) so answers are based on retrieved document passages.

• Divide the work across five focused agents for query handling, retrieval, answering, clarification, and conversational context.

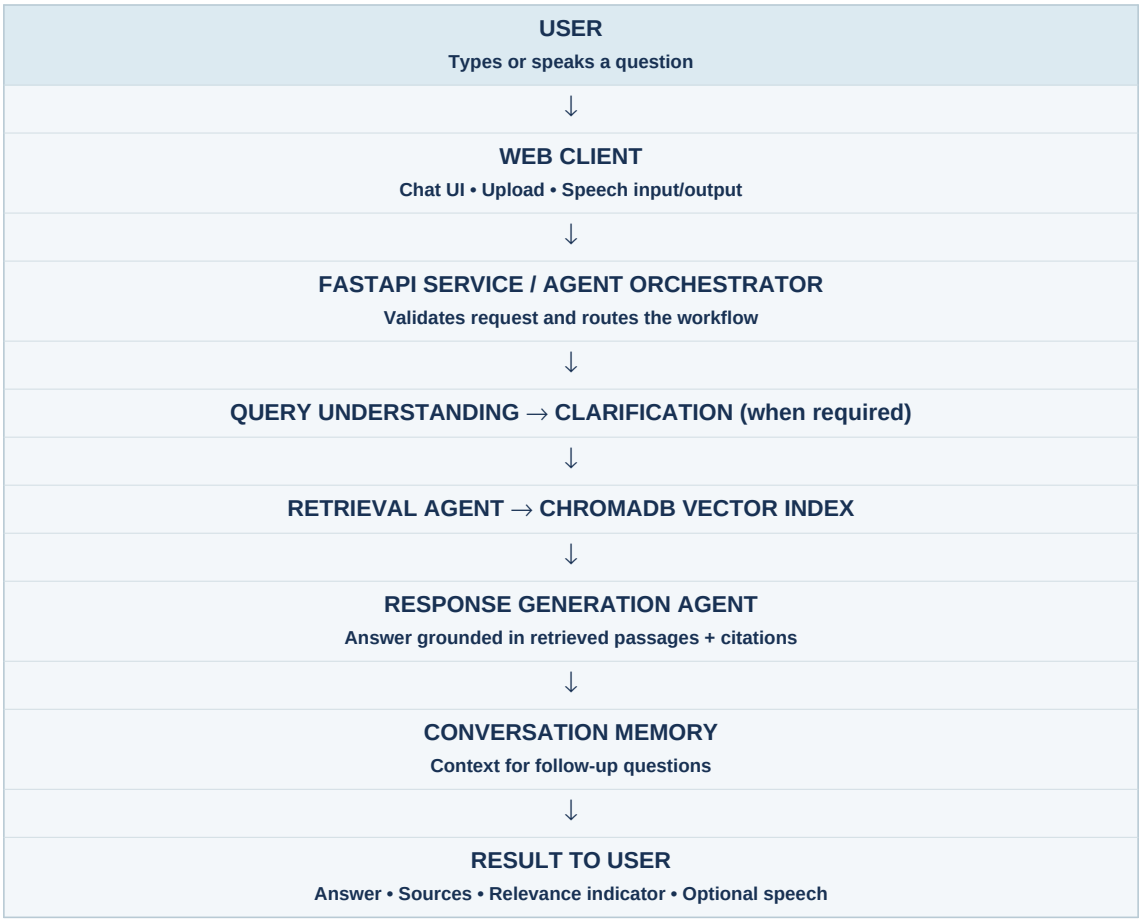
• Display source references and a carefully labelled relevance/confidence indicator.

• Support typed and voice-based questions where the browser supports speech features.

• Record unanswered questions to help identify gaps in the uploaded knowledge base.

3. High-Level Architecture

The main request path and the document preparation path are separated. This keeps file processing independent from question answering and allows the document index to be reused across multiple queries.



Document ingestion flow: Uploaded file → text extraction → chunking with overlap → embedding generation → vector index in ChromaDB, with metadata retained for source tracking.

4. Proposed Technology Choices

Layer	Proposed option	Purpose
Frontend	React or Streamlit	Chat screen, file upload, answer display and speech controls.
API / backend	Python + FastAPI	Receives requests, processes files and coordinates agent steps.
Text extraction	PyMuPDF, python-docx, CSV/TXT readers	Extracts readable text from supported formats.
Embeddings	all-MiniLM-L6-v2	Converts chunks and queries into vectors for semantic matching.
Vector store	ChromaDB	Stores vectors and retrieves similar passages.
Language model	Gemini API or a suitable open-source instruction model	Creates a response using the question and retrieved evidence.
RAG orchestration	LangChain (optional)	Can simplify retrieval and prompt pipeline integration.
Voice	Browser Web Speech API	Speech recognition and spoken output where supported.
Analytics	SQLite + basic charts	Tracks query activity, relevance indicators and unanswered topics.

These are initial research recommendations rather than fixed project constraints. Before implementation, confirm model access, cost, data privacy, deployment requirements, and browser compatibility.

5. RAG Processing Workflow

Step 1. Upload and parse: read the supported file and extract its text and page or section details.

Step 2. Chunk the content: split long text into manageable, slightly overlapping passages to preserve context.

Step 3. Create embeddings: encode each passage as a numerical vector and store it with document metadata.

Step 4. Retrieve evidence: encode the user question, search for semantically similar passages, and rank the matches.

Step 5. Generate a response: provide the question and selected passages to the language model with instructions to stay grounded in the evidence.

Step 6. Return traceable output: show the answer with source locations. If the evidence is insufficient, clearly state that the system cannot verify the answer.

6. Agent Responsibilities

Query Understanding Agent

Classifies the question type and identifies unclear wording, missing details, or the likely information need.

Clarification Agent

Asks a concise follow-up when the original question is too broad or ambiguous to answer reliably.

Retrieval Agent

Searches the vector index, selects relevant chunks, and passes source metadata to later stages.

Response Generation Agent

Writes a clear answer using the retrieved evidence and attaches references to the supporting document sections.

Conversation Memory Agent

Keeps useful recent turns so that follow-up prompts such as “What about exceptions?” can be interpreted in context.

7. Core Data Structures

Record	Example fields
	document_id, filename, file_type, upload_time, domain
Chunk	chunk_id, document_id, text, page_or_section, embedding_id
	conversation_id, user_id (optional), created_at
Message	message_id, conversation_id, role, text, timestamp
	query_id, answer, cited_chunk_ids, relevance_scores, status

Indicator design: A displayed confidence value should be described as retrieval relevance unless it has been calibrated and evaluated as a measure of answer correctness. Similarity scores alone do not prove that an answer is correct.

8. Example User Journey

A college uploads its student handbook. A student asks, “What is the attendance requirement?” The query is interpreted, relevant passages are retrieved, and the answer is generated with a reference to the handbook page.

If the student then asks, “Are there any exceptions?”, recent conversation context helps the system connect the follow-up to the attendance policy.

9. Testing and Evaluation

• Build a small set of questions with known answers for each document collection.

• Measure whether relevant source passages appear near the top of retrieval results.

• Check that generated answers are supported by the cited passages.

• Verify that ambiguous questions trigger clarification instead of unsupported assumptions.

• Test follow-up questions to ensure context is carried forward correctly.

• Include unsupported questions, different file formats, voice input, and failure cases in testing.

10. Suggested Eight-Week Plan

Period	Planned work
	Finalize architecture; implement upload, parsing, chunking, embeddings and vector indexing; test with two sample domains.
Weeks 3–4	Build query interpretation, retrieval ranking, answer generation and agent coordination.
	Add clarification, conversation context, source display and browser-based voice features.
Weeks 7–8	Add basic analytics and knowledge-gap tracking; evaluate at least three domains; fix issues and prepare the demo.

11. Development and Deployment Strategy

Agile development is a practical way to deliver the system in increments. Each iteration can end with a small working feature set, tests, and feedback from the mentor or intended users. This is a development method; cloud hosting is a separate decision.

Possible AWS deployment (if required)

• Amazon S3 can store uploaded documents.

• Amazon EC2 or AWS App Runner can host the FastAPI service.

• Amazon Bedrock may provide access to supported foundation models, depending on region and model availability.

• Amazon RDS or DynamoDB can store metadata, chat records, and analytics.

• ChromaDB can be retained for an initial prototype; a managed vector-search option can be assessed if production needs justify it.

Recommended starting point

Begin locally with Python, FastAPI, a basic browser interface, a sentence-transformer embedding model, ChromaDB, and an available language model. Implement and test the end-to-end RAG flow before adding optional voice, analytics, or cloud deployment. This keeps the first milestone focused and limits unnecessary setup.

12. Risks and Practical Considerations

• Document extraction quality can vary, especially for scanned or poorly formatted PDFs.

• Retrieval quality depends on chunk size, overlap, embedding choice, and the quality of the source documents.

• Answers must not be presented as verified when the retrieved evidence is weak or conflicting.

• Speech features differ by browser and may require user permission.

• Cloud services and external model APIs require checks for cost, privacy, credentials, and regional availability.

13. Conclusion

The proposed Intelligent Query Resolution System combines document ingestion, semantic retrieval, language-model generation, and five specialized agents to make knowledge in uploaded files easier to access. Source references and conservative handling of unsupported questions are central to trustworthiness. A small working RAG prototype should be the first deliverable; additional capabilities can be introduced in later iterations.

Note: This is a revised draft based on the supplied project material. Confirm the final architecture, technology choices, and milestone expectations with your internship mentor before submission.